

Root Without Reach: Capability-Directed Containers, Tracing, and MicroVMs in a Restricted Linux Sandbox

Sandbox Insights contributors

10 September 2026

Abstract

Linux sandboxes used for automated agents and hardened CI can present a process as uid 0 with every capability bit set while independently withholding identity mappings, selected syscall numbers, writable paths, socket operations, device nodes, and hardware virtualization. This work develops a capability-directed model from staged probes, failure reduction, tracing experiments, rootfs construction, and software-emulated guest execution. The central result is that "privilege" must be replaced by a vector over seven planes: namespace identity, syscall filtering, path policy, network policy, device exposure, resource ceilings, and observation/virtualization backends. The model explains why clone can create a namespace that mount cannot populate, why chown returns EINVAL rather than EPERM, why TCP and UDP calls using the same syscall family receive different decisions, why a loaded KVM driver does not imply a usable descriptor, and why ptrace denial does not preclude system-call tracing. The work records several techniques that are costly to rediscover: invalid-argument errno discrimination; stage-by-stage capability probes; ownership-neutral OCI extraction with a metadata sidecar; tracee-context dirfd resolution; a four-leg seccomp-notification supervisor; in-process use of namespace privileges that disappear across exec; appended-cpio device repair; marker-framed serial execution; and guest-clock snapshot measurement. These techniques yield one architecture with a shared image pipeline, three execution routes, and a dual-backend observability plane. The repository is Markdown-first and includes a local evidence ledger, captured observations, numbered audits, derived data, and secondary paper exports.

Keywords: Linux sandbox, user namespaces, seccomp user notification, Landlock, QEMU TCG, container runtime, system-call tracing

1. Introduction

The familiar Linux privilege vocabulary is too coarse for restricted execution environments. A process may print uid 0 and a full capability mask while being root only in a user namespace with one mapped identity. A seccomp filter may deny the legacy syscall for an operation while admitting a newer interface. A path-aware LSM may reject one pathname and accept another even when DAC metadata is identical. A cgroup program may permit UDP bind, deny TCP bind, reject loopback connect, and permit an external address. Host drivers may be present while their device nodes are absent and uncreatable. Each condition can surface as EPERM, EACCES, EINVAL, ENOENT, a signal, or a misleading higher-level diagnostic.

The investigation began by separating container compatibility, syscall observation, and microVM execution into the individual kernel operations they require. Identity maps, syscall entry, path verdicts, socket addresses, device acquisition, file ceilings, and observation backends were varied independently. The resulting negative cases were retained because the most useful mechanisms appeared where a plausible shortcut failed.

The contribution is both diagnostic and architectural. The project preserves negative results, records time-varying policy, derives a fail-closed route selector, and separates captured observation from local derivation and unbuilt design. The repository makes the distinction enforceable through a local evidence ledger with OBSERVED, CODE, DERIVED, DESIGN, LIMIT, and OPEN classes.

2. Evidence corpus and method

The factual base is docs/observations.md. Each entry retains the operation, result, discriminator, limitation, and reproduction status required to interpret it. Kernel-specific observations are marked CAPTURED; the current Windows authoring host can audit their local anchors but cannot present that audit as a fresh Linux execution. Derived inputs, including the six benchmark samples and policy-transition rows, are stored under experiments/data/.

The experiment contract is strict: an experiment is numbered, inputs are local and versioned, conditions are printed, negative results are retained, and exit codes distinguish matched, contradicted, and could-not-run outcomes. A producing exit code is captured before output formatting. A successful wrapper, generated file, or internal handler is not accepted as proof that the requested user-visible operation completed.

Four audits enforce the paper's local contract. Experiment 10 checks the required agent-first layout, 0BSD text, and absence of embedded external revision IDs. Experiment 20 resolves every evidence-ledger path, literal anchor, class, and condition inside this repository. Experiment 30 checks agreement across the Markdown model, observation notebook, architecture, and policy data. Experiment 40 parses the local benchmark CSV, verifies the common checksum, and recomputes the native-to-TCG interval.

Three methodological rules dominate. First, measure an operation rather than a nominal privilege. Second, isolate causes by removing or changing one mechanism at a time. Third, audit code, documentation, tests, and user-visible execution as separate claim surfaces. These rules were not ceremonial: the investigation

found probes that overwrote their own binaries, wrapper exits unrelated to the named syscall, a documented option missing from command-line parsing, valid artifacts produced before a tracer died with status 139, and policy claims that changed within one day.

3. A seven-plane environment model

Table 1 replaces the single word "restricted" with independently observable planes.

Plane	Evidence	Engineering consequence
Identity and namespace (N)	uid/gid maps, namespace ownership, setgroups	capabilities apply only relative to named namespace owners; unmapped IDs do not exist
Syscall filter (F)	pre-entry errno for a syscall number	legacy and modern interfaces can diverge; outer ERRNO can hide inner notification
Path policy (M)	verdict varies by pathname beyond DAC	detached mounts and mapped-owner paths can still be denied
Network policy (W)	verdict varies by protocol/address/port	sockaddr-aware cgroup hook; transport must be selected empirically
Device exposure (D)	host driver, node, fd, ioctl stages	hardware presence does not imply tenant reachability
Resource ceiling (R)	file, space, inode, time, process limits	mechanism tests can fail before reaching the mechanism
Virtualization/observation (V)	KVM, TCG, ptrace, user notification, proc memory	backend availability is compositional, not Boolean

The N/F/M decomposition is the base. N is a user namespace mapping only container uid/gid 0 to host 1000, with setgroups denied and a mount namespace owned by that user namespace. F is a syscall-number filter over unshare, setns, legacy mount operations, ptrace, and process_vm access. M is a path policy consistent with Landlock: it explains why mapped-owner directories with otherwise identical mount state receive different write decisions and why move_mount or detached-tree access fails after mount creation succeeds.

The observation matrix adds W, D, and R. TCP bind is denied for loopback, wildcard, and host addresses while UDP and Unix bind work; loopback TCP connect is denied while a host-LAN closed port reaches ECONNREFUSED; external port 80 changed from denied to allowed during the observation window. Seccomp cannot dereference sockaddr, so the combined signature is consistent with cgroup-BPF SOCK_ADDR hooks. KVM and TUN drivers were registered on the host but their nodes were not exposed. A 500 MiB per-file limit killed a VM launcher with SIGXFSZ, while a 64 MiB temporary filesystem caused misleading write loss.

V spans both execution and observation. KVM is unreachable, but executable memory permits TCG. Ptrace is denied, but a tracee can install a seccomp notification filter on itself. Read access to /proc/<pid>/mem can remain when process_vm_readv and proc-memory writes are denied. The correct question is therefore not "is tracing allowed?" but whether listener creation, handoff, argument read, result injection, and desired attachment semantics each work.

4. Diagnostic techniques

4.1 Invalid-argument errno discrimination

A seccomp filter evaluates a syscall before its body and cannot dereference a path pointer. Calling a disputed syscall with a deliberately nonexistent path, impossible PID, or closed fd provides a cheap discriminator. ENOENT, ESRCH, or EBADF shows that argument-specific in-kernel validation ran; unconditional EPERM suggests a pre-entry filter. Kernel control-flow analysis and positive controls are still required because a downstream LSM can also return EPERM.

This technique separated filtered mount from LSM-denied move_mount, filtered process_vm_readv from executing pidfd operations, and syscall policy from network policy. Its general form is valuable: choose two arguments that only the suspected mechanism can distinguish, hold all other state fixed, and retain the control in the permanent probe.

4.2 Probe creation, attachment, and use separately

The new mount API demonstrated why feature probes must follow stage boundaries. At one policy epoch, filesystem contexts and detached mounts could be created; attachment through move_mount failed; opening paths through the detached tree failed separately. Later the outer policy began denying fsopen and open_tree while clone3 namespace creation remained available. A single "mount supported" bit would be both uninformative and temporally wrong.

The same decomposition applies elsewhere. For KVM: driver registration, node visibility, node creation, descriptor transfer, and usable ioctls are distinct. For a seccomp supervisor: listener, notification delivery, argument read, and fd injection are distinct. For a VM: kernel artifact, VMM, accelerator, console, network, and lifecycle are distinct.

4.3 Defeat witness-specific artifacts

Two small probes had misleading results. Character device 0:0 is Linux's whiteout device and bypasses the ordinary device-node capability check; a real device number must be paired with it. A Go process is multithreaded before main, so unshare(CLONE_NEWUSER) returns EINVAL even when policy allows the operation; a single-threaded helper or clone is required. More generally, every mutating probe runs in a disposable child, missing fixtures produce SKIP rather than DENIED, and wrapper exit status must derive from the named operation.

4.4 Namespace ownership and procs provenance

The clone-family filter gap allowed a process to create a user and network namespace that it owned. In-process raw netlink then created a veth pair, assigned addresses and a /24 netmask, raised interfaces, and exchanged an ICMP echo; the same operation in the parent namespace returned EPERM. However, uid_map writes failed because authorization followed the user namespace that had mounted procs, not the newly created namespace. With no mapped uid, exec dropped the child's namespace-granted capabilities. The practical rule is unusual and important: when privilege exists only in a namespace and cannot survive exec, link the operation into the creating process; do not shell out to an equivalent helper.

5. Rootfs and compatibility engineering

Image acquisition generally survived because it is HTTPS and file I/O. Image extraction crossed the identity plane. OCI layers commonly contain files such as `etc/shadow` owned by gid 42; when the user namespace maps only gid 0, restoring that owner returns `EINVAL`. GNU tar, a Go extractor, a container storage layer, and package-manager privilege drops all reached the same wall.

The robust design extracts without applying unmapped ownership and records the intended uid, gid, mode, applied values, and reason in a path-keyed sidecar. This is not a lossy `--no-same-owner` workaround: the sidecar supports diagnostics, re-export, and an ownership-interposition layer. It still changes no kernel DAC decision and must be reported as virtual metadata.

Ownership neutrality does not excuse an incomplete OCI extractor. Layers must be applied in order; whiteouts and opaque directories must be interpreted after each layer; links must be preserved; and every resolved path must remain under the destination, including through a symlink introduced earlier in the same layer. A particularly useful regression image creates a self-referential cache symlink and removes it with a later whiteout; plain tar leaves the loop behind.

Entering a prepared root has a strict order: validate the root; open all host descriptors needed for `stdio`, `logs`, `PTYs`, or exposed devices; `chroot` and `chdir`; resolve the executable inside the new root; then `exec`. Resolving too early can select a host binary, while recomputing storage paths after an environment reset can create an empty second root and produce a false "binary missing" error.

Libc interposition is a compatibility plane, not isolation. Path virtualization and ownership virtualization are separate jobs. A complete path mapper resolves the `*at` family through the tracee's `dirfd`s and reverse-maps `getcwd`, `readlink`, `readdir`, and `realpath`. It still cannot see static programs, Go's direct syscalls, or arbitrary raw-syscall code. A route selector can reject known-ineligible ELF payloads, but it cannot prove coverage from ELF metadata alone.

6. Observation without ptrace

The tracing design turns `ptrace` denial from a terminal limitation into a backend selection problem. The tracee enables no-new-privileges, installs a `seccomp` user-notification filter over its own syscalls, passes the listener descriptor to its parent with `SCM_RIGHTS`, and `execs`. The supervisor receives syscall number and six arguments while the tracee is blocked. It reads memory through `/proc/<pid>/mem`, continues ordinary calls, or emulates selected operations. For `open/openat` it opens in supervisor context and injects the real descriptor with `NOTIF_ADDFD`.

The backend has four required legs: listener creation, listener handoff and a real notification, argument reads, and `result/fd` injection. A successful listener alone is not sufficient. If argument reads disappear while a handler falls back to `CONTINUE`, an enforcement or dry-run feature can report success while the real syscall executes. This is the clearest reason to probe once and refuse the tier rather than degrade per call.

Tracing must preserve the tracee's execution context. A tracee's numeric `dirfd` is not the same descriptor in the tracer. Resolve it

through `/proc/<pid>/fd`, and resolve `AT_FDCWD` through the tracee's `cwd`. Scan inherited descriptors after successful `exec` and after `close-on-exec` processing. Record continued calls with unknown results rather than fabricated success. Escape every tracee-controlled JSON string and include `begin`, `syscall`, `signal`, `exit`, and summary events.

Semantic interventions need their own limits. Hiding `TracerPid` and `wchan` should overwrite values with equal-length zero sequences so buffer offsets do not move; vector reads require a seam window across `iovecs`. Suppressing `exit_group` may expose later behavior, but falling off an exit path can fault or loop. The tracer should retain the suppressed status for the immediate fault, record the signal, bound attempts per process, and apply an external timeout.

Backend parity must be tested at the user-visible stream, not inferred from a listener or internal handler. Required fixtures cover normal exit, signal death, successful `exec`, unknown continued results, descriptor injection, and summary closure. Any disagreement among implementation, tests, command help, and Markdown remains `OPEN` until one end-to-end oracle passes.

7. MicroVM workflows under software emulation

The device investigation established KVM unavailability as a chain rather than inferring it from `ENOENT`. It tested every reachable node view, real-device `mknod`, `devtmpfs` creation, handle-based `open`, descriptor donation, and `ring-0`-adjacent alternatives. Each path had a distinct kernel or namespace gate. This justified selecting QEMU TCG instead of continuing to patch KVM-dependent managers.

TCG preserved much of the microVM workflow. A pinned Alpine guest reached userspace in approximately 1.6 s on the microvm machine. Eight 128 MiB guests booted in 1.97 s total, or 4.06 boots/s. QEMU migration produced a 76 MB snapshot that resumed into three independent clones. With TCP listeners denied, two `slirp` instances exchanged guest traffic through UDP host forwards. An in-guest package installation supplied GCC and compiled the same benchmark used on the host.

The benchmark's checksum was 165be307 everywhere. Host runs measured 721.4 and 754.8 Mops/s; `chroot` runs measured 845.9 and 866.8; `microvm-Alpine` measured 35.1 twice. Cross-products of the same-day native/`chroot` and guest values span 20.55 to 24.70, supporting a rounded 20-25x compute cost for this workload, not a general TCG factor. Guest types clustered at 31-35 Mops/s, so TCG dominated the machine and distribution differences.

Two boot details are easy to rediscover expensively. The tested `initramfs` contained an empty `/dev`, so PID 1 had no console; appending a raw `cpio` archive with `/dev/console` repaired it without host `mknod`. The same artifacts produced kernel `printk` but not userspace serial output on QEMU's microvm machine, while the `pc` machine was interactive. The resulting profiles are deliberately different: `microvm` for `printk`-oriented fleets, `pc` with ACPI disabled for a serial control plane.

The serial protocol holds both FIFO ends open read-write before QEMU starts, waits for a readiness marker, verifies a nonce handshake, and frames each command with random line-anchored `begin/end` markers plus an exit code. Retry feeding is permitted only during lossy bootstrap. Snapshot latency is

measured by guest uptime at snapshot and the first post-resume tick, avoiding unrelated host startup/log latency while exposing the tick-resolution bound.

Under TCG, a guest kernel is separate from the host kernel but executes inside the QEMU userspace process. This is an emulator-process boundary, not hardware-enforced virtualization. It is stronger than chroot's path switch and independent of the shared-kernel namespace gap, but its security ultimately depends on QEMU and the outer sandbox.

8. Unified architecture

The architecture yields one front end with a common content-addressed rootfs store, a live capability probe, three route engines, and one observability plane. Requests declare required semantics: zero/shared-kernel/separate-kernel security; host/private/guest networking; image root versus real mounts; one-shot/managed/ snapshot lifecycle; and native-required versus emulation-allowed performance. The selector reports the achieved route, probe epoch, degradations, refusals, and surfaces it does not provide. A weaker route never satisfies a stronger security request.

The image pipeline fetches by digest, preflights blocks, inodes, and file-size limits, extracts safely with whiteouts and ownership sidecar, completes the environment idempotently, and then adapts the same directory tree. The chroot route uses it directly. The shared-kernel route adds only currently proven namespace, seccomp, and LSM controls. The microVM route packs it as initramfs, adds a device archive and init protocol, and boots it under a measured QEMU profile. The interposer consumes the same metadata sidecar.

Request	Selected route	Required disclosure
trusted, maximum-speed build	chroot	no process, network, mount, or kernel isolation
native-speed confinement	shared-kernel	exact current controls; policy belongs to outer operator
hostile workload or snapshot/fleet	QEMU/TCG	emulator-process boundary and measured compute cost
syscall observation	ptrace or seccomp notification	backend limits and outer-filter blind spots
cooperative dynamic compatibility	libc interposition	incomplete raw-syscall coverage; no security property

Control follows measured network policy: Unix sockets and FIFOs for local control, serial framing for guests, and UDP for guest data when TCP bind is unavailable. Lifecycle readiness is event-driven through pidfds, readiness descriptors, or nonce markers; fixed sleeps never establish state. Logs and evidence are opened before root changes and stored outside small temporary filesystems.

Implementation order begins with the corpus and probe, then the image store and extractor, chroot execution, lifecycle, shared-kernel controls, TCG execution, fleet/snapshot/network, tracing, ownership interposition, and packaging. Each milestone has a positive and negative witness, an end-to-end driven pass, and three review lenses. This order front-loads the two components most likely to invalidate everything downstream: environment classification and image extraction.

9. Limitations and threats to validity

The captured observations cover a small number of sandbox instances on one host class. They cannot generalize policy, performance, or backend availability to other environments. The path-dependent verdict matrix is repeatable as a local record but still lacks a clean Linux reconstruction that enables and disables the candidate policy while holding other LSMs constant.

Policy was demonstrably time-varying. New mount syscalls changed from permitted to denied and external port 80 changed from denied to allowed. The paper treats these as transitions, not timeless properties. Tool selection must re-probe on boot, kernel, host, or canary changes.

The 20-25x TCG interval is derived from one compute benchmark, two native samples, two chroot samples, and two microVM samples. It does not cover memory bandwidth, compilation, syscall density, or I/O. No same-host KVM baseline exists inside the sandbox. Snapshot timing is bounded by a two-second guest tick.

The unified runtime is a design and has not passed its milestone gates. Libc ownership interposition, seccomp-notify backend parity, 32-bit tracees, PTY behavior before chroot, and full tracing overhead on extraction storms remain open.

The review process itself has residual risk. Literal drift checks prove that a local passage still exists, not that it is correct. Captured observations can contain instrumentation defects, missing alternatives, or over-broad wording. Assume more errors remain and require fresh conditions for every new execution.

10. Conclusion

Root-shaped identity does not imply usable root operations. The correct unit of reasoning is a capability vector, measured at the operation and stage that a tool actually needs. Once identity mappings, syscall filters, path policies, network hooks, device exposure, resource ceilings, and observation backends are separated, apparently contradictory results become predictable and actionable.

The capability model changes the design response. No single weakened container mode should be presented as universal. A common image pipeline can feed a fast chroot compatibility route, a native shared-kernel confinement route, and a QEMU/TCG route with a separate guest kernel and VM lifecycle. The route is chosen from requested semantics and current probes, while a ptrace/seccomp-notify observability plane records what actually occurred. Degradation is explicit, machine-readable, and never allowed to satisfy an isolation requirement.

The durable contribution is not a product claim but a set of instruments and mechanisms: invalid-argument controls, staged probes, ownership sidecars, tracee-context resolution, self-installed notification filters, in-process namespace work, cpio device overlays, framed serial execution, and guest-clock snapshot timing. Each exists because a plausible shortcut failed. Keeping those techniques beside their conditions is how the project avoids paying for the same discoveries again.